EXPERIMENT – 6

AIM: To create a stored procedure transfer_employee(emp_id, new_dept_id)with validation and error handling, and to implement database triggers for salary validation and audit logging. The procedure and triggers are tested using normal and edge-case inputs.

SOURCE CODE:

USING EXPERIMENT – 3 DATABASE

SQL QUERIES:

STORED PROCEDURE: transfer_employee

The procedure will:

1. Check whether the employee exists.
2. Check whether the new department exists.
3. Check whether the employee is already in that department.
4. Transfer the employee.
5. Handle errors using SQLEXCEPTION.
6. Roll back the transaction if an error occurs.

```
DELIMITER //

CREATE PROCEDURE transfer_employee( IN
        p_emp_id INT,
        IN p_new_dept_id INT
) BEGIN
        DECLARE v_employee_count INT DEFAULT 0; DECLARE
        v_department_count INT DEFAULT 0; DECLARE v_current_dept INT
        DEFAULT 0;

        -- Error handler
        DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN
                ROLLBACK;
                SELECT 'Error occurred. Transaction rolled back.' AS
message;
        END;

        START TRANSACTION;
```

```
        -- Check whether employee exists SELECT COUNT(*)
        INTOv_employee_count FROM
        Employee
        WHERE emp_id = p_emp_id;

        IF v_employee_count = 0 THEN ROLLBACK;
                SELECT 'Error: Employee does not exist.' AS message;

        ELSE

                -- Check whether department exists SELECT COUNT(*)
                INTOv_department_count FROM
                Department
                WHERE dept_id = p_new_dept_id;

                IF v_department_count = 0 THEN ROLLBACK;
                        SELECT 'Error: Department does not exist.' AS


message;


                ELSE

                        -- Get current department SELECT dept_id
                        INTOv_current_dept FROM
                        Employee
                        WHERE emp_id = p_emp_id;

                        -- Check if already in requested department IF v_current_dept =
                        p_new_dept_id THEN
                                ROLLBACK;
                                SELECT 'Error: Employee is already in this AS message;
department.'
                        ELSE

                                -- Transfer employee UPDATE
                                Employee
                                SET dept_id = p_new_dept_id WHERE emp_id
                                = p_emp_id;

                                COMMIT;

                                SELECT
                                        'Employee transferred successfully.' AS message;
```

```
                ENDIF; END

            IF;

        END IF;

END // DELIMITER;
```

TESTING THE STORED PROCEDURE

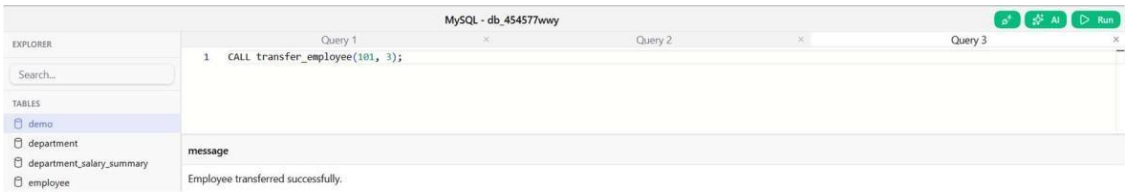
```
SELECT
        e.emp_id,
        e.emp_name,
        d.dept_name
FROM Employee e
JOIN Department d
        ON e.dept_id = d.dept_id WHERE
e.emp_id = 101;
```

OUTPUT:




```
CALL transfer_employee(101, 3);
```

OUTPUT:



```
SELECT
        e.emp_id,
        e.emp_name,
        d.dept_name
FROM Employee e JOIN
Department d
        ON e.dept_id = d.dept_id WHERE
e.emp_id = 101;
```

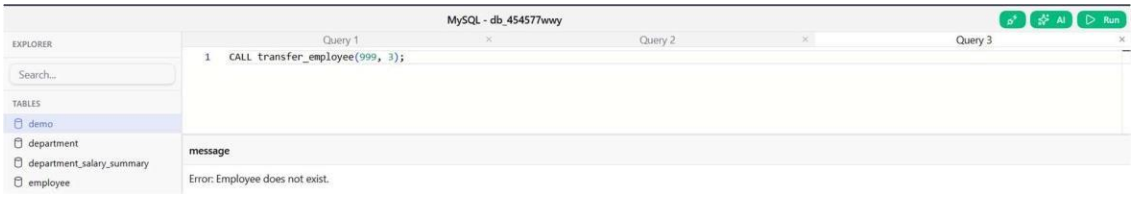
OUTPUT:



EDGE CASE – EMPLOYEE DOES NOT EXIST

```
CALL transfer_employee(999, 3);
```

OUTPUT:



EDGE CASE – DEPARTMENT DOES NOT EXIST

```
CALL transfer_employee(101, 99);
```

OUTPUT:



EDGE CASE – SAME DEPARTMENT

CALL transfer_employee(101, 3);

OUTPUT:



SALARY VALIDATION TRIGGER

Let, salary must be >= 15000

DELIMITER //

CREATE TRIGGER validate_employee_salary BEFORE INSERT ON
Employee
FOR EACH ROW
BEGIN

IF NEW.salary < 15000 THEN SIGNAL SQLSTATE
'45000'
SET MESSAGE_TEXT = 'Salary must be at least 15000'; END IF;

END // DELIMITER;

TEST SALARY VALIDATION

INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id) VALUES
(131, 'Test Employee', 'M', 10000, '2026-09-25', 1);

OUTPUT:

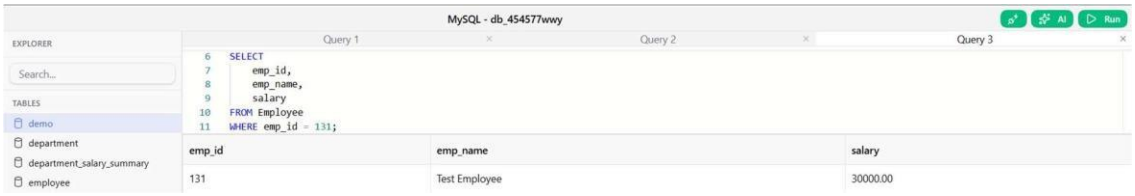


TEST VALID SALARY

```
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id) VALUES
(131, 'Test Employee', 'M', 30000, '2026-09-25', 1);

SELECT
        emp_id,
        emp_name,
        salary
FROM  Employee  WHERE
emp_id = 131;
```

OUTPUT:



SALARY VALIDATION FOR UPDATE

```
DELIMITER //

CREATE TRIGGER validate_employee_salary_update BEFORE UPDATE ON
Employee
FOR EACH ROW
BEGIN

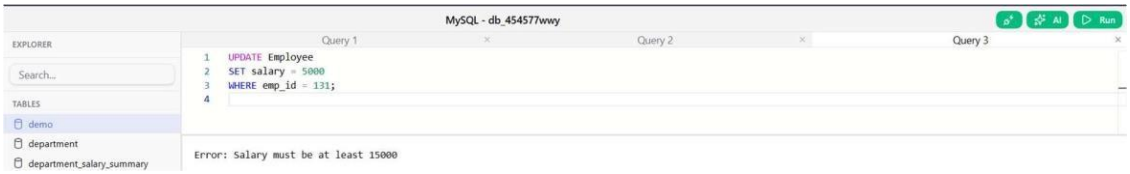
        IF NEW.salary < 15000 THEN SIGNAL SQLSTATE
                '45000'
                SET MESSAGE_TEXT = 'Salary must be at least 15000'; END IF;

END // DELIMITER ;
```

TEST INVALID SALARY UPDATE

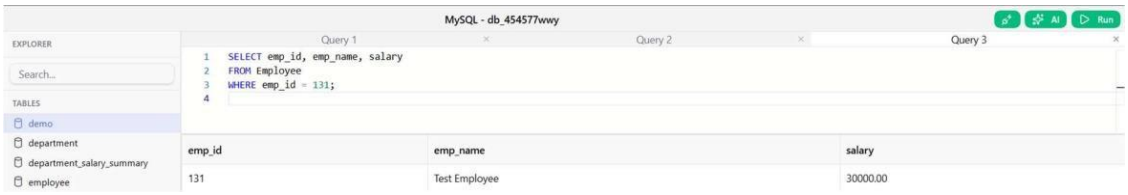
```
UPDATE Employee SET
salary = 5000
WHERE emp_id = 131;
```

OUTPUT:




```
SELECT emp_id, emp_name, salary FROM
Employee
WHERE emp_id = 131;
```

OUTPUT:



CREATE AUDIT TABLE

```
CREATE TABLE Salary_Audit (
        audit_id INT AUTO_INCREMENT PRIMARY KEY,
        emp_id INT,
        old_salary DECIMAL(10,2), new_salary
        DECIMAL(10,2),
        changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        action VARCHAR(30)
);
```

AUDIT LOGGING TRIGGER

```
DELIMITER //

CREATE TRIGGER salary_audit_trigger AFTER UPDATE
ON Employee
FOR EACH ROW
BEGIN
```

```
    IF OLD.salary <> NEW.salary THEN

        INSERT INTO Salary_Audit (
            emp_id, old_salary,
            new_salary, action
        )
        VALUES (
            NEW.emp_id,
            OLD.salary, NEW.salary,
            'SALARYUPDATED'

        );

    ENDIF;

END //

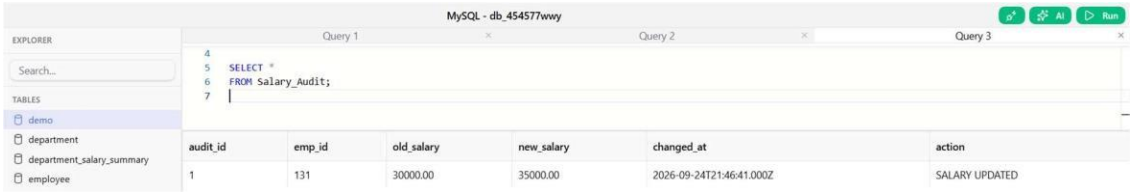
DELIMITER;
```

TEST AUDIT TRIGGER

```
UPDATE Employee SET
salary = 35000
WHERE emp_id = 131;

SELECT *
FROM Salary_Audit;
```

OUTPUT:



TEST MULTIPLE SALARY CHANGES

```
UPDATE Employee SET
salary = 40000
WHERE emp_id = 131;

UPDATE Employee SET
salary = 45000
WHERE emp_id = 131;

SELECT
        audit_id, emp_id,
        old_salary,
        new_salary, action
FROM Salary_Audit WHERE
emp_id = 131;
```

OUTPUT:



TEST EDGE CASE – SALARY UNCHANGED

```
UPDATE Employee SET
salary = 45000
WHERE emp_id = 131;
```

No new audit record is created

TEST EDGE CASE – NULL SALARY

```
DROP TRIGGER validate_employee_salary; DELIMITER //

CREATE TRIGGER validate_employee_salary BEFORE INSERT ON
Employee
FOR EACH ROW
BEGIN
```

```
    IF NEW.salary IS NULL OR NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Salary cannot be NULL and must be at least 15000'; END IF;

END // DELIMITER;

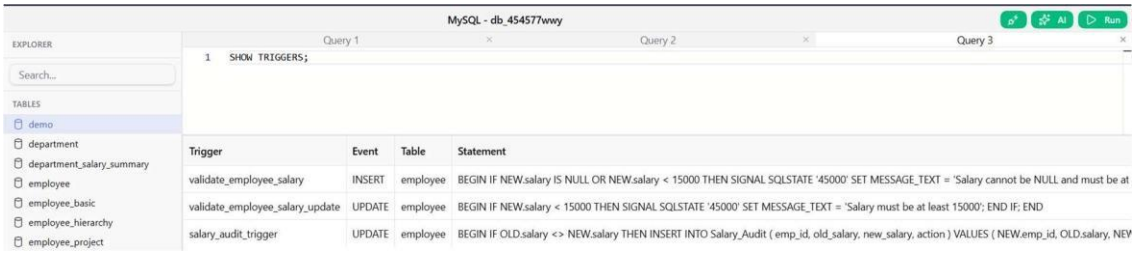
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id) VALUES
(132, 'Null Salary', 'F', NULL, '2026-09-25', 1);
```

OUTPUT:



CHECK ALL TRIGGERS SHOW

TRIGGERS; **OUTPUT**



CHECK STORED PROCEDURE

```
SHOW PROCEDURE STATUS
WHERE Db = 'db_454577wwy';
```

OUTPUT:

